

ArtBridge API 명세서 (코드 기준 v0.8)

라우트 파일 43개(`src/app/api/**/route.ts`) · HTTP 메서드 핸들러 56개(파일당 GET/POST 등 복수 메서드 포함). 기준: bigfive2026/cadet-project main. 인증은 Auth.js(JWT) 세션.

프로그램(Program) 도메인 API

문서화한 라우트 파일은 6개, 총 엔드포인트(HTTP 메서드 export)는 11개다. 모든 행은 `src/app/api/programs/**/route.ts` 와 그 서비스(`src/lib/*`)·검증 스키마(`src/lib/validation/*`)의 실제 코드를 근거로 한다.

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
GET	/api/programs	공개 프로그램 목록 (deletedAt IS NULL + 공개 상태)	공개	쿼리: <code>category?</code> (문자열)	<code>200</code> Program 배열 (<code>creatorProfile{id, studioName, profileImageUrl}</code> 포함)	없음(항상 200)	구현완료 (<code>listPublicPrograms</code> Prisma findMany)
POST	/api/programs	프로그램 생성 (status는 일정 기반 자동 선정, 기본 RECRUITING)	로그인필요 → CREATOR(본인 creatorProfile 필요)	body(zod <code>programCreateSchema</code>): <code>title</code> (1~200, 필수), <code>description?</code> (≤5000), <code>priceKrw</code> (int≥0, 필수), <code>category?</code> (≤100), <code>startDate?</code> / <code>endDate?</code> / <code>recruitDeadline?</code> (date), <code>maxParticipants?</code> (양의 int)	<code>201</code> 생성된 Program	<code>401</code> 비로그인 / <code>400</code> JSON 파싱 실패·검증 실패 / <code>403</code> 비크리에이터·프로필 없음	구현완료 (<code>createProgram</code> → Prisma create)
POST	/api/programs/ai-suggest	AI 가격·혜택·주차 구성 추천 (OpenAI 또는 Mock 폴백)	로그인필요 → CREATOR	body(zod): <code>description</code> (1자 이상, 필수), <code>duration?</code> , <code>category?</code> , <code>targetAudience?</code>	<code>200</code> <code>{suggestedPrice, benefits[], programStructure[{week, title, description}], reason, source:"openai" "mock"}</code>	<code>401</code> 비로그인 / <code>403</code> 비크리에이터 / <code>400</code> JSON·검증 실패 / (서비스 예외 시에도 <code>200 {error}</code> 반환, 키 정보 비노출)	구현완료 + Mock폴백 (<code>suggestProgram</code> : 키 없음·OpenAI 실패 시 결정론적 <code>suggestMock</code>)
GET	/api/programs/{id}	공개 상세 조회 (soft-delete 제외)	공개	path: <code>id</code>	<code>200</code> Program 상세 (<code>creatorProfile{id, studioName, profileImageUrl}</code> 포함)	<code>404</code> 미존재·soft-delete	구현완료 (<code>getProgramDetail</code> Prisma findFirst)
PATCH	/api/programs/{id}	프로그램 수정 (일정 변경 시 status 재계산, CLOSED 전이 시 알림)	로그인필요 → CREATOR + 본인 소유	body(zod <code>programUpdateSchema</code> , 최소 1개 필드): <code>title?</code> , <code>description?</code> (nullable), <code>priceKrw?</code> , <code>category?</code> (nullable), <code>startDate?</code> / <code>endDate?</code> / <code>recruitDeadline?</code> (nullable date), <code>maxParticipants?</code> (nullable)	<code>200</code> 수정된 Program	<code>401</code> 비로그인 / <code>400</code> JSON·검증 실패(필드 0개 포함) / <code>403</code> 비크리에이터·타인 소유 / <code>404</code> 미존재·삭제됨	구현완료 (<code>updateProgram</code> → <code>loadOwnedProgram</code> 소유권검증 + Prisma update)
DELETE	/api/programs/{id}	프로그램 soft delete (deletedAt 설정, 물리삭제 금지)	로그인필요 → CREATOR + 본인 소유	path: <code>id</code>	<code>200</code> <code>{ok:true}</code>	<code>401</code> 비로그인 / <code>403</code> 비크리에이터·타인 소유 / <code>404</code> 미존재·삭제됨	구현완료 (<code>deleteProgram</code> → soft delete)
POST	/api/programs/{id}/complete	완료 처리(크리에이터 일괄 납품 요청, SPEC-013 에스스로 재정의)	로그인필요 → CREATOR + 본인 소유	path: <code>id</code> (body 없음)	<code>200</code> <code>{programStatus, requestedDeliveries, notifiedParticipants}</code>	<code>401</code> 비로그인 / <code>404</code> 미존재·삭제됨 / <code>403</code> 비소유자 / <code>400</code> IN_PROGRESS 아님 (서비스 결과 status: 400/403/404)	구현완료 (<code>completeProgram</code> : PAID·미요청 참여 일괄 <code>deliveryRequestedAt</code> + 알림 트랜잭션)
POST	/api/programs/{id}/applications	프로그램 참여 신청 (무료=즉시 ACCEPTED, 유료=PENDING_PAYMENT/결제)	로그인필요 (FAN/CREATOR 모두, 자기 프로그램은 CREATOR 역할로 신청 불가)	path: <code>id</code> ; body(zod <code>applySchema</code>): <code>message?</code> (≤1000)	<code>201</code> <code>{id, status, paymentId, settlementId, amount, provider, merchantUid, paymentParams}</code>	<code>401</code> 비로그인 / <code>400</code> JSON·검증 실패·모집중 아님·기한 경과·자기프로그램 / <code>404</code> 프로그램·크리에이터프로필 없음 / <code>409</code> 중복 신청·정원 초과 / <code>500</code> 트랜잭션 실패	구현완료 (<code>applyToProgram</code> : 정원·중복·결제 provider 트랜잭션)
GET	/api/programs/{id}/applications	크리에이터 본인 프로그램의 신청 목록 (최신순)	로그인필요 → CREATOR	path: <code>id</code>	<code>200</code> ProgramApplication 배열 (<code>user{id, name}</code> 포함)	<code>401</code> 비로그인 / <code>403</code> 비크리에이터	구현완료 (<code>listApplicationsForCreator</code> Prisma findMany)
GET	/api/programs/{id}/reviews	프로그램 리뷰 목록 + 평균 평점(크리에이터가 받은 평점만)	공개	path: <code>id</code>	<code>200</code> <code>{reviews[{id, rating, comment, tags, createdAt, revieweeId, user, reviewee}], avgRating}</code>	없음(미존재 프로그램은 빈 목록·avg null)	구현완료 (<code>listProgramReviews</code> Prisma findMany + 집계)
POST	/api/programs/{id}/reviews	리뷰 작성 (양방향: 팬→크리에이터 / 크리에이터→팬)	로그인필요 (완료 승인된 참여자/소유 크리에이터)	path: <code>id</code> ; body(zod <code>reviewSchema</code>): <code>rating</code> (int 1~5, 필수), <code>comment?</code> (≤1000, 공백→null), <code>tags?</code> (최대 5개·각 ≤20자·중복제거), <code>revieweeId?</code>	<code>201</code> <code>{id, rating, comment, tags}</code>	<code>401</code> 비로그인 / <code>400</code> JSON·검증 실패·revieweeId 누락(크리에이터)·자기평가·미완료 / <code>403</code> 비참여자·완료 안 된 대상 / <code>404</code> 프로그램·소유자 없음 / <code>409</code> 중복 리뷰 / <code>500</code> 생성 실패	구현완료 (<code>createReview</code> : 자격·중복 검증 + Prisma create, P2002 처리)

비고: 표의 `{id}` 는 Next App Router의 `[id]` 동적 세그먼트다. `/api/programs/{id}/complete` 의 인증·역할은 라우트에서 비로그인(401)만 직접 차단하고, 나머지 CREATOR·소유권(403/404/400)은 `completeProgram` 서비스 결과 status 로 위임된다(applications·reviews POST도 동일 패턴). AI 추천은 서비스 예외 시에도 키 노출 방지를 위해 200으로 응답한다.

프로그램 신청·에스크로 (Application)

전수 분석한 라우트 파일(전부 실제 코드 근거):

- `src/app/api/programs/[id]/applications/route.ts` (POST, GET)
- `src/app/api/applications/[id]/route.ts` (PATCH)
- `src/app/api/applications/[id]/contract/route.ts` (POST)
- `src/app/api/applications/[id]/request-delivery/route.ts` (POST)
- `src/app/api/applications/[id]/approve-completion/route.ts` (POST)

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
(참조)	/api/programs/{id}/applications	참여 신청(POST)·신청목록 조회(GET) — 경로상 프로그램 하위라 위 프로그램 도메인 표에 상세 기재	—	—	—	—	중복 제거(프로그램 도메인 참조)
PATCH	/api/applications/{id}	신청 취소(cancel, 팬 본인) 또는 확정 참여자 제외(remove, 크리에이터). PAID면 환불·정산 ON_HOLD 처리	로그인필요 (cancel=신청자 본인 / remove=프로그램 소유 CREATOR)	<code>processSchema : action</code> ("cancel" "remove"), <code>removedReason?</code> (string, ≤500)	200 + <code>{ id, status }</code>	401 / 400(Invalid JSON·검증실패·취소불가상태·미수락참여) / 403(타인 취소·비CREATOR·비소유자) / 404(신청없음) / 500(트랜잭션실패)	구현완료 (<code>cancelProgramApplication / removeProgramParticipant @ src/lib/applications.ts</code> , 환불·정산 보류·알림 트랜잭션)
POST	/api/applications/{id}/contract	ACCEPTED 신청에 대한 계약 생성/조회(역등, terms 스냅샷)	로그인필요 (팬 본인 또는 프로그램 소유 CREATOR)	본문 없음 (id = ProgramApplication.id)	200 + <code>{ id }</code> (Contract.id)	401 / 400(미ACCEPTED 신청) / 403(계약 비참여자) / 404(신청없음)	구현완료 (<code>getOrCreateContract @ src/lib/contracts.ts</code>)
POST	/api/applications/{id}/request-delivery	납품 요청(크리에이터 → PAID 참여, IN_PROGRESS 한정, 역등). 팬에게 DELIVERY_REQUESTED 알림	로그인필요 + 프로그램 소유 CREATOR	본문 없음	200 + <code>{ deliveryRequestedAt, notifiedFan }</code>	401 / 400(미IN_PROGRESS·이미완료·미결제) / 403(비소유자) / 404(신청없음)	구현완료 (<code>requestDelivery @ src/lib/reviews.ts</code> , 상태가드·결재확인·트랜잭션·알림)
POST	/api/applications/{id}/approve-completion	에스크로 완료 승인(팬=지불자). Payment/Settlement RELEASED + completionApprovedAt + 전원 승인 시 Program COMPLETED, COMPLETION_APPROVED·MUTUAL_REVIEW_REQUESTED 알림	로그인필요 (해당 참여 지불자=팬 본인)	본문 없음	200 + <code>{ completionApprovedAt, programStatus, releasedPayment, releasedSettlement, mutualReviewRequested }</code>	401 / 400(납품요청 선행 없음) / 403(지불자 아님) / 404(신청없음) / 500(Settlement 누락 등 트랜잭션 롤백)	구현완료 (<code>approveCompletion @ src/lib/reviews.ts</code> , Settlement 무결성 @MX:ANCHOR 가드·단일 트랜잭션)

비고:

- 모든 라우트는 `getCurrentUser()` (`@lib/auth`)로 세션 기반 인증. 미로그인 시 일괄 401.
- 역할 판정은 라우트(GET의 CREATOR 체크)와 서비스 계층(`ensureCreator / resolveAccess` /지불자 일치)에서 이중으로 수행됨.
- "Mock폴백"은 결제 provider에 국한된 동작이며, 신청·에스크로 도메인 로직 자체는 실 Prisma 트랜잭션으로 모두 구현완료. 부분구현/스텝/순수 Mock인 엔드포인트는 없음.

계약·서명·결제 (Contract)

전수 대상: `/api/contracts/[id]` 하위 6개 route.ts. 각 라우트는 단일 HTTP 메서드만 export(PATCH 5개 + POST 1개). 모든 인증은 `getCurrentUser()` (세션) 기반이며, 서비스 컨텍스트로 `{ userId, role, creatorProfileId }` 를 전달한다. 역할 게이트는 라우트가 아니라 서비스 계층(`resolveAccess` 또는 `userId` 직접 비교)에서 수행한다. 모든 서비스는 `src/lib/contracts.ts` 에 실제 Prisma 트랜잭션으로 구현됨 → 라우트 단위 구현상태는 모두 [구현완료]. 단, 결제(payment)는 결제 Provider가 환경변수 미설정 시 `MockPaymentProvider` 로 폴백되어 외부 네트워크 없이 즉시 PAID 처리하는 [Mock폴백] 경로를 포함한다.

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
PATCH	/api/contracts/{id}/amount	합의 금액 제시(크리에이터가 양의 정수 금액 제시, <code>agreedAmount</code> 갱신 + <code>terms.amountProposedAt</code> 기록 + 팬에게 <code>CONTRACT_AMOUNT_PROPOSED</code> 알림)	로그인필요 / CREATOR(프로그램 소유자만, <code>resolveAccess===creator</code>)	<code>amountProposeSchema : { amount: number(int, positive) }</code>	200 <code>{ agreedAmount: number }</code>	401(비로그인) / 400(Invalid JSON, amount 비양수·비정수) / 403(소유 크리에이터 아님) / 404(계약 없음) / 409(팬 서명 후 재조율 불가)	구현완료
PATCH	/api/contracts/{id}/amount/agree	합의 금액 동의(팬 본인이 제시 금액 동의, <code>terms.amountAgreedAt</code> 기록. 미제시·거부 상태면 400, 이미 동의 시 역등 반환)	로그인필요 / FAN(팬 본인만, <code>userId===application.userId</code>)	<code>amountAgreeSchema : { agreed: true }</code> (literal 강제)	200 <code>{ amountAgreedAt: string(ISO) }</code>	401 / 400(Invalid JSON, agreed!==true, 금액 미제시) / 403(팬 본인 아님) / 404(계약 없음)	구현완료

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
PATCH	/api/contracts/{id}/amount/reject	합의 금액 거부·결렬(팬 본인. 단일 트랜잭션으로 신청 REJECTED + terms.amountRejectedAt + 프로그램 CONTRACTING→RECRUITING 복귀 + 크리에이터에게 CONTRACT_AMOUNT_REJECTED 알림)	로그인필요 / FAN(팬 본인만)	<code>amountRejectSchema : { agreed: false } (literal 강제)</code>	200 <code>{ applicationStatus: "REJECTED", programStatus: string }</code>	401 / 400(Invalid JSON, agreed!==false, 금액 미제시) / 403(팬 본인 아님) / 404(계약 없음) / 409(서명 후 결렬 불가) / 500(트랜잭션 실패)	구현완료
PATCH	/api/contracts/{id}/sign	팬 전자 서명(팬 본인. 금액 제시했으나 미합의의 시행 등 요구. fanSignedAt 설정(역등). 양측 서명 완료 시 CONTRACT_SIGNED 알림)	로그인필요 / FAN(팬 본인만, resolveAccess==fan)	<code>signSchema : { agreed: true } (literal 강제)</code>	200 <code>{ fanSignedAt: Date }</code>	401 / 400(Invalid JSON, agreed!==true, 제시 금액 미합의 상태) / 403(팬 아님) / 404(계약 없음)	구현완료
PATCH	/api/contracts/{id}/sign/creator	크리에이터 전자 서명(프로그램 소유 크리에이터. creatorSignedAt 설정(역등). 양측 서명 완료 시 CONTRACT_SIGNED 알림)	로그인필요 / CREATOR(프로그램 소유자만, resolveAccess===creator)	<code>signSchema : { agreed: true } (literal 강제)</code>	200 <code>{ creatorSignedAt: Date }</code>	401 / 400(Invalid JSON, agreed!==true) / 403(소유 크리에이터 아님) / 404(계약 없음)	구현완료
POST	/api/contracts/{id}/payment	결제 시작(팬 본인. 양측 서명 완료 가드 후 agreedAmount 기준 결제. Mock 폴백이면 즉시 PAID 트랜잭션(Payment+Settlement+Program IN_PROGRESS+알림), sandbox면 PENDING Payment 생성 후 결제창 메타 반환)	로그인필요 / FAN(팬 본인만, userId===application.userId)	<code>paymentSchema</code> (선택, 본문 비어도 허용): <code>{ provider?: "mock" }</code>	"portone"	"toss"(default "mock") — 단 서비스는 파싱된 provider를 무시하고 resolvePaymentProvider() (환경변수) 사용	200 <code>{ paymentId, settlementId: string }</code>

비고:

- 라우트 처리 순서는 공통적으로 비로그인 401 → JSON 파싱 실패 400 → zod 검증 실패 400 → 서비스 결과(403/404/409/500) → 200.
- `verifyPayment` (결제 검증·확정) 서비스 함수는 `src/lib/contracts.ts` 에 구현돼 있으나, `src/app/api/contracts/` 하위에는 이를 노출하는 콜백/검증 route.ts가 존재하지 않음(이 디렉터리 전수 기준 미노출). GET 메서드를 export하는 라우트는 이 디렉터리에 없음(계약 생성·조회 `getOrCreateContract` 도 contracts 디렉터리 밖에서 호출됨).
- `paymentSchema` 의 `provider` 필드는 검증만 통과시키고 실제 분기에 미사용 → 서버 권위 검증(클라이언트 provider 불신뢰) 의도.

결제 콜백(Payment)

대상 디렉터리 `/src/app/api/payments` 전수 조사 결과, route.ts는 `callback/route.ts` 단 1개이며 `POST` / `GET` 두 메서드를 export한다. 두 메서드 모두 동일하게 `verifyPayment()` 서비스(`src/lib/contracts.ts`)로 위임한다.

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
POST	/api/payments/callback	PG 결제 검증·확정. JSON 본문 <code>{merchantUid, providerTxId}</code> 수신 → PG 단건 조회로 금액·주문번호·상태 대조 후 성공 시에만 PAID 확정 (정산 PENDING 생성 + 프로그램 IN_PROGRESS + 알림 생성). 먹등 처리(PAID/RELEASED 재요청 시 200).	로그인필요 (<code>getCurrentUser</code> , 없으면 401). 추가로 서비스단에서 해당 계약의 팬 본인만 허용 (<code>ctx.userId !== payment.fanUserId</code> → 403). 별도 role 분기는 없음.	zod <code>paymentCallbackSchema : merchantUid: string().min(1) , providerTxId: string().min(1) (body는 request.json() 파싱)</code>	200: <code>{ paymentId, status, programStatus }</code>	401 (비로그인), 400 (Invalid JSON body / Validation failed / Payment verification failed= 금액·상태·주문번호 불일치), 403 (팬 본인 아님), 404 (Payment not found), 500 (트랜잭션 실패)	구현완료 (Mock폴백 포함) — <code>verifyPayment</code> 는 Prisma <code>\$transaction</code> 으로 payment 갱신·settlement 생성·program 갱신·notification 생성을 실제 수행. <code>resolvePaymentProvider()</code> 가 PG 키 부재 시 <code>MockPaymentProvider</code> (서버 보관값 신뢰), <code>PAYMENT_PROVIDER=portone</code> +키 존재 시 실제 <code>fetch("https://api.portone.io/...")</code> 사용. P2002 unique 위반은 먹등 PAID로 처리.
GET	/api/payments/callback	PG 리다이렉트 쿼리 파라미터 형태 수용. <code>?merchantUid=&providerTxId=</code> (또는 <code>imp_uid</code> 별칭)을 읽어 동일하게 검증·확정. POST와 완전히 동일한 <code>verifyPayment</code> 흐름.	로그인필요 (<code>getCurrentUser</code> , 없으면 401). 서비스단 팬 본인 검증(403) 동일.	쿼리스트링: <code>merchantUid , providerTxId</code> (없으면 <code>imp_uid</code> fallback) → zod <code>paymentCallbackSchema</code> 로 동일 검증	200: <code>{ paymentId, status, programStatus }</code>	401 (비로그인), 400 (Validation failed=빈 파라미터), 403 (팬 본인 아님), 404 (Payment not found), 500 (트랜잭션 실패). ※ GET에는 Invalid JSON body 경로 없음	구현완료 (Mock폴백 포함) — POST와 동일 서비스 (<code>verifyPayment</code>) 공유, 위 구현 근거 동일.

근거 파일:

- 라우트: `/Users/youngkwang/projects/artbridge/cadet-project/src/app/api/payments/callback/route.ts`
- 검증 스키마: `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/validation/contract.ts` (`paymentCallbackSchema`)
- 서비스: `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/contracts.ts` (`verifyPayment` , L612-702)
- Provider: `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/payment/provider.ts` (`resolvePaymentProvider` , `MockPaymentProvider` , `SandboxPaymentProvider.verifyPayment`)

작품 주문·배송 (ArtworkOrder)

대상 디렉터리 `/src/app/api/artwork-orders` 하위 라우트 전수 분석. 모든 라우트는 `getCurrentUser()` 로 로그인 검증 후 `@/lib/artwork-fulfillment` 의 서비스 함수에 위임하며, 인가(역할·소유권·상태 전이) 검증은 서비스 계층에서 수행한다. 서비스 5종 모두 실제 Prisma `$transaction` + 알림 생성으로 동작하고 Mock 폴백 분기가 없어 전부 구현완료.

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
POST	/api/artwork-orders/{id}/shipment	작품 주문 발송 처리(운송장 등록, 주문 SHIPPED, 구매자 알림)	로그인필요 → 서비스에서 CREATOR + 작품 소유자만 허용	<code>shipArtworkOrderSchema</code> : carrier(1~20), trackingNo(4120), shippedAt?(date)	<code>{ orderId, status, shipmentId }</code> (200)	400(JSON파싱실패/검증실패/PAID·PREPARING 외 상태), 401(미인증), 403(CREATOR 아님·소유자 아님), 404(주문없음), 500(트랜잭션실패)	구현완료
POST	/api/artwork-orders/{id}/received	구매자 수령 확정(주문 RECEIVED, 배송 deliveredAt, 정산 AVAILABLE 전환, 알림)	로그인필요 → 서비스에서 주문의 fanUserId(구매자) 본인만 허용	본문 없음 (params의 id만 사용)	<code>{ orderId, status, settlementStatus? }</code> (200)	401(미인증), 403(구매자 아님), 404(주문없음), 400(SHIPPED·DELIVERED 외 상태), 500(트랜잭션실패)	구현완료
POST	/api/artwork-orders/{id}/issues	구매자 배송/상품 이슈 신고(이슈 생성, 주문 ISSUE_OPENED, 정산 ON_HOLD, 작가 알림)	로그인필요 → 서비스에서 주문의 fanUserId(구매자) 본인만 허용	<code>reportArtworkIssueSchema</code> : type(enum: NOT_DELIVERED/DAMAGED/WRONG_ITEM/NOT_AS_DESCRIBED/REFUND_REQUEST/OTHER), message(1~2000), imageUrl?(url,≤500)	<code>{ issueId, orderStatus }</code> (201)	400(JSON파싱실패/검증실패/CANCELLED·REFUNDED 상태), 401(미인증), 403(구매자 아님), 404(주문없음), 500(트랜잭션실패)	구현완료
POST	/api/artwork-orders/{id}/issues/resolve	작가의 이슈 해결 처리(OPEN·REVIEWING 이슈 RESOLVED, 주문 RECEIVED, 정산 AVAILABLE 전환, 구매자 알림)	로그인필요 → 서비스에서 CREATOR + 작품 소유자만 허용	<code>resolveArtworkIssueSchema</code> : resolutionNote?(≤1000) ※서비스에서 미사용 (<code>_input</code>)	<code>{ orderId, status, resolvedIssueCount, settlementStatus? }</code> (200)	400(JSON파싱실패/검증실패/CANCELLED·REFUNDED 상태), 401(미인증), 403(CREATOR 아님·소유자 아님), 404(주문없음), 409(열린 이슈 없음), 500(트랜잭션실패)	구현완료
POST	/api/artwork-orders/{id}/refund	작가의 환불 처리(주문 REFUNDED, 결제 REFUNDED, 정산 ADJUSTED·차감조정, 재고 복원, 구매자 알림)	로그인필요 → 서비스에서 CREATOR + 작품 소유자만 허용	<code>refundArtworkOrderSchema</code> : reason(trim, 1~500)	<code>{ orderId, status, paymentStatus?, settlementStatus? }</code> (200)	400(JSON파싱실패/검증실패/CANCELLED·REFUNDED·RECEIVED 상태), 401(미인증), 403(CREATOR 아님·소유자 아님), 404(주문없음), 500(트랜잭션실패)	구현완료

근거 메모

- 라우트 파일 5개 모두 HTTP 메서드 export는 `POST` 단일 (GET/PATCH/PUT/DELETE 없음). 누락 메서드 없음.
- 인증: 라우트는 `getCurrentUser()` null 검사로 401만 처리하고, 역할/소유권 검증(403)·상태 검증(400)·존재 검증(404)·중복/충돌(409)은 전부 `src/lib/artwork-fulfillment.ts` 서비스 내부에서 반환.
- 검증 스키마 출처: `src/lib/validation/artwork-order.ts` (received는 스키마 없이 body 미파싱).
- `received` · `issues` (신고)는 구매자(fanUserId) 권한, `shipment` · `refund` · `issues/resolve` 는 CREATOR+소유자 권한.
- 구현상태 판정: 5개 서비스 함수 모두 `prisma.$transaction` 으로 실제 DB 갱신·알림 생성 수행, Mock/스텝 분기 없음 → 전부 구현완료.

크리에이터 (작품·정산·업로드)

전체 6개 라우트 파일, 8개 엔드포인트(HTTP 메서드 기준)를 전수 문서화했다. 모든 라우트는 `getCurrentUser()` (실제 세션→DB 조회, `include: { creatorProfile: true }`)로 인증하며, `user.role === "CREATOR"` 및 `user.creatorProfile` 존재를 공통 가드로 검사한다. 모든 데이터 접근은 실제 Prisma 쿼리 또는 실제 파일시스템 쓰기이며, 어떤 라우트에도 Mock 폴백이 존재하지 않는다.

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
GET	/api/creator/settlements	본인(크리에이터)의 정산 내역 조회. 멤버십/포스트/계약/선착순프로그램/작품 결제 정산을 Settlement 기준으로 최신순 통합 조회	CREATOR (로그인 + role=CREATOR + creatorProfile 필수)	없음 (쿼리/바디 없음)	<code>{ settlements: Settlement[] }</code> (각 항목에 payment + fan/membership/post/contract/programApplication/artworkOrder 포함, status 200)	401 미로그인, 403 CREATOR 아님	구현완료 (<code>listCreatorSettlements</code> in <code>src/lib/queries/contracts.ts</code> , 실제 Prisma <code>settlement.findMany</code>)
POST	/api/creator/uploads	크리에이터 에셋 이미지 업로드. <code>public/uploads/creator-assets/</code> 에 파일 저장 후 공개 URL 반환	CREATOR	multipart/form-data, 필드 <code>file</code> (File). 허용 타입 jpeg/png/webp/gif, 최대 5MB	<code>{ url: "/uploads/creator-assets/{파일명}", contentType, size }</code> (status 201)	401 미로그인, 403 CREATOR 아님, 400 multipart 파싱 실패/file 누락/미허용 타입, 413 5MB 초과	구현완료 (라우트 내 <code>fs.mkdir / writeFile</code> 직접 처리, 외부 서비스 없음)
GET	/api/creator/works	본인 활동/이력(CreatorWork) 목록 조회 (startedAt desc, createdAt desc)	CREATOR	없음	<code>CreatorWork[]</code> (배열 직접 반환, status 200)	401 미로그인, 403 CREATOR 아님	구현완료 (라우트 내 <code>prisma.creatorWork.findMany</code>)
POST	/api/creator/works	활동/이력(CreatorWork) 생성	CREATOR	JSON, <code>creatorWorkCreateSchema</code> : title(필수, 1~120), kind?(≤80), description?(≤2000), imageUrl?(URL 또는 /uploads/creator-assets/ 경로), externalUrl?(URL), startedAt?/endedAt?(date, endedAt≥startedAt)	생성된 <code>CreatorWork</code> (status 201)	401 미로그인, 403 CREATOR 아님, 400 JSON 파싱 실패/검증 실패(issues 포함)	구현완료 (<code>creatorWorkCreateSchema</code> in <code>src/lib/validation/artwork.ts</code> + <code>prisma.creatorWork.create</code>)

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
PATCH	/api/creator/works/{id}	본인 소유 CreatorWork 부분 수정	CREATOR + 소유자 본인	JSON, <code>creatorWorkUpdateSchema</code> : 전 필드 optional(최소 1개 필수), kind/description/imageUrl/externalUrl는 빈 문자열→null clearable, startedAt/endedAt clearable date	수정된 <code>CreatorWork</code> (status 200)	401 미로그인, 403 CREATOR 아님 또는 타 크리에이터 소유, 400 JSON 파싱/검증 실패, 404 work 없음	구현완료 (소유권 확인 후 <code>prisma.creatorWork.update</code>)
DELETE	/api/creator/works/{id}	본인 소유 CreatorWork 삭제(하드 딜리트)	CREATOR + 소유자 본인	없음 (경로 파라미터 id)	<code>{ ok: true, deleted: true }</code> (status 200)	401 미로그인, 403 CREATOR 아님 또는 타 소유, 404 work 없음	구현완료 (<code>prisma.creatorWork.delete</code>)
GET	/api/creator/artworks	본인 판매 작품(Artwork) 목록 조회 (createdAt desc)	CREATOR	없음	<code>Artwork[]</code> (배열 직접 반환, status 200)	401 미로그인, 403 CREATOR 아님	구현완료 (라우트 내 <code>prisma.artwork.findMany</code>)
POST	/api/creator/artworks	판매 작품(Artwork) 생성	CREATOR	JSON, <code>artworkCreateSchema</code> : title(필수, 1~120), description?(≤2000), imageUrl?(URL 또는 업로드 경로), priceKrw(필수, 양의 정수), stock(정수≥0, 기본 1), status(DRAFT/PUBLISHED/HIDDEN, 기본 DRAFT)	생성된 <code>Artwork</code> (status 201)	401 미로그인, 403 CREATOR 아님, 400 JSON 파싱/검증 실패	구현완료 (<code>artworkCreateSchema</code> + <code>prisma.artwork.create</code>)
PATCH	/api/creator/artworks/{id}	본인 소유 Artwork 부분 수정	CREATOR + 소유자 본인	JSON, <code>artworkUpdateSchema</code> : 전 필드 optional(최소 1개 필수), title 1~120, description/imageUrl clearable, priceKrw 양의 정수, stock≥0, status enum	수정된 <code>Artwork</code> (status 200)	401 미로그인, 403 CREATOR 아님 또는 타 소유, 400 JSON 파싱/검증 실패, 404 artwork 없음	구현완료 (소유권 확인 후 <code>prisma.artwork.update</code>)
DELETE	/api/creator/artworks/{id}	본인 소유 Artwork 삭제. 주문(ArtworkOrder) 이력 있으면 하드 딜리트 대신 status=HIDDEN 소프트 처리	CREATOR + 소유자 본인	없음 (경로 파라미터 id)	주문 없을 때 <code>{ ok: true, deleted: true, hidden: false }</code> / 주문 있을 때 <code>{ ok: true, deleted: false, hidden: true, artwork }</code> (status 200)	401 미로그인, 403 CREATOR 아님 또는 타 소유, 404 artwork 없음	구현완료 (<code>artworkOrder.count</code> 분기 후 <code>update(HIDDEN)</code> 또는 <code>delete</code>)

비고: `uploads/route.ts` 는 413(파일 크기 초과)을 반환하는 유일한 라우트다. 500은 어느 라우트에서도 명시적으로 반환하지 않으며, Prisma/fs 예외 시 Next.js 런타임이 암묵적 500을 발생시킨다. PUT 메서드를 export하는 라우트는 없다(수정은 모두 PATCH).

멤버십(Membership)

대상 라우트 4개 파일, HTTP 메서드 export 총 5개를 전수 문서화했습니다. 모든 행은 실제 코드 근거 기준입니다.

Method	Endpoint	기능	인증 역할	Request	Response	주요 오류	구현상태
POST	/api/membership-plans	멤버십 플랜 생성 (크리에이터 전용)	로그인필요 + CREATOR (<code>getCurrentUser</code> → role===“CREATOR” → <code>creatorProfile</code> 존재)	<code>membershipPlanCreateSchema</code> (validation/membership.ts): <code>title</code> (string 1~100, 필수), <code>priceKrw</code> (int 양수, 필수), <code>description</code> (string ≤500, 선택)	201, 생성된 <code>membershipPlan</code> 레코드 전체 (prisma create 반환)	401(비로그인), 403(비CREATOR / creatorProfile 없음), 400(잘못된 JSON / Zod 검증 실패)	구현완료 (prisma.membershipPlan.create 실DB)
POST	/api/membership-plans/ai-suggest	멤버십 AI 가격·혜택 추천	로그인필요 + CREATOR (<code>getCurrentUser</code> → role===“CREATOR”)	inline zod 스키마: <code>description</code> (string min1, 필수), <code>category</code> (string, 선택), <code>targetAudience</code> (string, 선택)	200, <code>MembershipSuggestResult</code> : <code>suggestedPrice</code> (int), <code>benefits</code> (string[]), <code>reason</code> (string), <code>source</code> (“openai” “mock”)	401(비로그인), 403(비CREATOR), 400(잘못된 JSON / Zod 검증 실패). 추천 생성 실패 시에도 status 200으로 <code>{error:“Failed to generate suggestion”}</code> 반환	Mock폴백 (<code>suggestMembership</code> : OPENAI_API_KEY 있고 호출 성공 시 OpenAI, 키 없음·오류·스키마위반 시 결정론적 <code>suggestMembershipMock</code> 폴백)
PATCH	/api/membership-plans/[id]	멤버십 플랜 수정 (본인 소유)	로그인필요 + CREATOR + 소유자 (role===“CREATOR” && creatorProfile 존재, 그리고 plan.creatorProfileId === 본인)	<code>membershipPlanUpdateSchema</code> (createSchema의 <code>.partial()</code>): <code>title</code> / <code>priceKrw</code> / <code>description</code> 모두 선택	200, 수정된 <code>membershipPlan</code> 레코드 전체 (prisma update 반환)	401(비로그인), 403(비CREATOR/creatorProfile 없음 / 타인 소유), 400(잘못된 JSON / Zod 검증 실패), 404(플랜 없음)	구현완료 (findUnique 소유검증 + prisma.membershipPlan.update 실DB)
DELETE	/api/membership-plans/[id]	멤버십 플랜 삭제 (본인 소유, 활성 멤버 없을 때만)	로그인필요 + CREATOR + 소유자 (role===“CREATOR” && creatorProfile 존재, plan.creatorProfileId === 본인)	본문 없음 (<code>_request</code> 미사용), 경로 파라미터 <code>id</code>	200, <code>{ ok: true }</code>	401(비로그인), 403(비CREATOR/creatorProfile 없음 / 타인 소유), 404(플랜 없음), 409(활성 멤버 보유 시 <code>prisma.membership.count > 0</code>)	구현완료 (소유검증 + membership.count 서버판정 + prisma.membershipPlan.delete 실DB)
PATCH	/api/memberships/[id]/cancel	멤버십 구독 취소 (본인 구독)	로그인필요 (<code>getCurrentUser</code> null → 401, 역할 무관). 소유권은 서비스 레이어 <code>cancelMembership</code> 에서 검증	본문 없음 (<code>_request</code> 미사용), 경로 파라미터 <code>id</code> (membershipId), <code>user.id</code> 로 소유 검증	200, <code>cancelMembership</code> 성공 data: <code>{ id, status, cancelledAt }</code> (status → “CANCELLED”)	401(비로그인), 404(멤버십 없음), 403(본인 구독 아님), 400(status !== “ACTIVE”), 500(update 실패) — status는 서비스 result.status 그대로 전달	구현완료 (<code>lib/membership.ts</code> cancelMembership: findUnique 검증 + prisma.membership.update 실DB, try/catch 500 처리)

근거 메모 (정확성 확인용)

- 인증 패턴:** 모든 라우트가 `@/lib/auth` 의 `getCurrentUser()` 를 사용. `requireUser` / `requireRole` 헬퍼는 이 라우트들에서 사용되지 않고, 각 핸들러 내부에서 직접 null/role/creatorProfile 체크.

- 취소 라우트 역할 차이: `/api/memberships/{id}/cancel` 은 CREATOR 역할 체크가 없음(라우트는 로그인만 확인). 실제 소유권 차단(403)·상태 차단(400)·존재 차단(404)·DB실패(500)은 `cancelMembership` 서비스가 반환하는 `result.status` 를 그대로 응답에 전달.
- AI 추천 폴백: 라우트의 catch가 실패 시에도 HTTP 200을 반환하도록 작성되어 있어(데모 무중단), 서비스 자체도 예외를 throw하지 않고 Mock으로 폴백. 따라서 이 엔드포인트의 구현상태는 Mock폴백.
- 활성 멤버 판정: DELETE는 `prisma.membership.count({ where: { planId: id } })` 로 서버 측 판정(@MX:WARN 주석으로 명시). cancel 서비스의 `isActiveMember` 는 별도 함수이며 이 4개 라우트에서는 직접 호출되지 않음.
- Membership 생성(가입/구독 시작) 라우트는 대상 두 디렉터리에 존재하지 않음 — `membership-plans` 와 `memberships/{id}/cancel` 만 존재. 멤버 가입 엔드포인트는 이 범위 내 코드에 없으므로 지어내지 않음.

관련 파일 경로:

- `/Users/youngkwang/projects/artbridge/cadet-project/src/app/api/membership-plans/route.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/app/api/membership-plans/ai-suggest/route.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/app/api/membership-plans/{id}/route.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/app/api/memberships/{id}/cancel/route.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/validation/membership.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/membership.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/ai/suggest.ts`

포스트·커뮤니티(Post/Community)

전수 대상 라우트 6개 / HTTP 메서드 export 7개. 모든 행은 실제 코드 근거.

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
POST	<code>/api/posts</code>	포스트 생성 (PUBLIC/MEMBER_ONLY/PAID)	로그인필요 + CREATOR(+creatorProfile 보유)	<code>postCreateSchema</code> : title(1~200), body(1+), visibility(PUBLIC MEMBER_ONLY PAID), priceKrw(양의 정수, 선택), status(DRAFT PUBLISHED, 선택). PAID면 priceKrw>0 필수(superRefine)	201 + 생성된 Post 객체(prisma.post.create 결과 전체)	401(비로그인), 403(CREATOR 아님), 403(creatorProfile 없음), 400(JSON 파싱 실패), 400(검증 실패: PAID인데 priceKrw 없음 포함)	구현완료 — prisma.post.create로 실제 영속화
POST	<code>/api/posts/{id}/comments</code>	포스트 댓글 작성	로그인필요 + canViewPost 통과(작성자/PUBLIC/활성멤버/구매자)	<code>postCommentCreateSchema</code> : body(trim, 1~1000)	201 + 생성된 PostComment(author.name include)	401(비로그인), 404(글 없음), 404(DRAFT인데 작성자 아님), 403(canViewPost 거부), 400(JSON 파싱 실패), 400(검증 실패)	구현완료 — prisma.postComment.create + canViewPost 실제 접근제어
POST	<code>/api/posts/{id}/likes</code>	포스트 좋아요 토글(없으면 추가, 있으면 취소)	로그인필요 + canViewPost 통과	본문 없음(<code>_request</code>). postId는 경로	200 + <code>{ liked: true }</code> (추가) 또는 <code>{ liked: false }</code> (취소)	401(비로그인), 404(글 없음), 404(DRAFT인데 작성자 아님), 403(canViewPost 거부)	구현완료 — postLike unique(postId,userId) 조회 후 create/delete 토글
POST	<code>/api/posts/{id}/purchase</code>	PAID 포스트 단건 구매 (Payment/Settlement/Notification 원자 생성)	로그인필요	<code>purchaseSchema</code> : provider(enum["mock"], default "mock"). 본문 선택값(누락 시 mock)	200 + <code>{ paymentId, settlementId, amount, feeKrw }</code>	401(비로그인), 400(JSON 파싱 실패), 400(검증 실패), 404(글 없음/DRAFT), 400(본인 글 구매), 400(PAID 아님), 400(priceKrw<=0), 409(중복 구매/P2002 경합), 500(charge 실패/트랜잭션 실패)	Mock폴백 — purchasePost가 mockPaymentProvider (MockPaymentProvider 고정 인스턴스, resolvePaymentProvider 미사용) 사용. charge 항상 성공, 외부 PG 미호출. DB 영속화 (Payment/Settlement/Notification)는 실제 수행
POST	<code>/api/community-posts</code>	커뮤니티 글 생성	로그인필요 + canAccessCommunity 통과(활성멤버/결제완료 참여자/소유 크리에이터)	<code>communityPostCreateSchema</code> : creatorProfileId(1+), title(1~200), content(1+)	201 + 생성된 CommunityPost(authorId=user.id)	401(비로그인), 400(JSON 파싱 실패), 400(검증 실패), 403(canAccessCommunity 거부)	구현완료 — prisma.communityPost.create + canAccessCommunity 실제 접근제어
PATCH	<code>/api/community-posts/{id}</code>	커뮤니티 글 수정(부분 수정)	로그인필요 + canManagePost(작성자 본인 또는 스튜디오 소유 CREATOR)	<code>communityPostUpdateSchema</code> : title(1~200, 선택), content(1+, 선택), 둘 중 최소 하나 필수(refine)	200 + 수정된 CommunityPost	401(비로그인), 404(글 없음), 403(권한 없음), 400(JSON 파싱 실패), 400(검증 실패)	구현완료 — prisma.communityPost.update로 실제 영속화
DELETE	<code>/api/community-posts/{id}</code>	커뮤니티 글 삭제	로그인필요 + canManagePost(작성자 본인 또는 스튜디오 소유 CREATOR)	본문 없음(<code>_request</code>). id는 경로	200 + <code>{ ok: true }</code>	401(비로그인), 404(글 없음), 403(권한 없음)	구현완료 — prisma.communityPost.delete로 실제 삭제

비고:

- `/api/posts` 라우트에는 목록 조회용 GET export가 없음(POST만 존재). `/api/community-posts` 도 POST만, `{id}` 는 PATCH/DELETE만 존재(GET/PUT 없음). 전수 확인 결과 위 7개 메서드가 전부.

- 인증은 모두 `getCurrentUser()` 기반(null → 401). 별도 requireRole/requireUser 헬퍼는 미사용이며, 역할 체크는 라우트 내 인라인(`user.role !== "CREATOR"`) 또는 access 헬퍼 (canViewPost/canAccessCommunity/canManagePost)로 수행.
- 구매 라우트의 Mock폴백은 의도된 안전 기본동작: `purchasePost` 가 환경변수 기반 `resolvePaymentProvider()` 가 아닌 고정 `mockPaymentProvider` 를 직접 호출하므로, PG 키 설정과 무관하게 항상 Mock charge로 처리됨.

근거: `src/app/api/{notifications,studio,auth}/**/*.route.ts` 전수 Read + 서비스(`src/lib/notifications.ts` , `src/lib/queries/notifications.ts` , `src/lib/auth.ts` , `src/auth.ts` , `src/lib/prisma.ts`) + 검증(`src/lib/validation/studio.ts`) 교차 확인. `prisma` 는 실 `PrismaClient` (Mock 아님)이며 세 도메인 모두 실제 DB 연동.

Notification (알림)

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
GET	<code>/api/notifications</code>	로그인 사용자의 알림 목록(최신순) 조회	로그인필요 (<code>getCurrentUser()</code> null→401)	본문 없음 (request 미사용)	<code>200</code> <code>Notification[]</code> (createdAt desc)	<code>401</code> Unauthorized	구현완료 — <code>listNotifications(user.id) → prisma.notification.findMany</code>
PATCH	<code>/api/notifications/{id}/read</code>	개별 알림 읽음 표시(본인 알림만)	로그인필요 (<code>getCurrentUser()</code> null→401)	path param <code>id</code> , 본문 없음	<code>200</code> <code>{ id }</code>	<code>401</code> Unauthorized / 서비스 결과 <code>result.status</code> (현재 항상 ok=true 반환)	구현완료 — <code>markNotificationRead : prisma.notification.updateMany({ where: {id,userId}, data:{readAt:now} })</code> (userId 일치로 소유권 격리)
PATCH	<code>/api/notifications/read-all</code>	사용자의 미읽음 알림 전체 읽음 처리	로그인필요 (<code>getCurrentUser()</code> null→401)	본문 없음	<code>200</code> <code>{ count }</code> (갱신 건수)	<code>401</code> Unauthorized	구현완료 — <code>markAllNotificationsRead : updateMany({ where:{userId, readAt:null}, data:{readAt:now} })</code> 후 <code>result.count</code> 반환

Studio (스튜디오)

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
PATCH	<code>/api/studio</code>	크리에이터 스튜디오(CreatorProfile) 정보 편집	CREATOR (null→401, role≠CREATOR→403, 비소유자→403)	<code>studioUpdateSchema</code> (zod): <code>creatorProfileId</code> (필수, min1), <code>studioName?</code> (1~80), <code>bio?</code> (≤500), <code>category?</code> (≤40), <code>coverImageUrl?</code> / <code>profileImageUrl?</code> / <code>instagramUrl?</code> / <code>websiteUrl?</code> (url 또는 ""→null 해제)	<code>200</code> 갱신된 CreatorProfile	<code>401</code> Unauthorized / <code>403</code> CREATOR 권한 없음 / <code>400</code> Invalid JSON body / <code>400</code> Validation failed(+issues) / <code>403</code> 소유 프로필 아님(<code>user.creatorProfile?.id !== body.creatorProfileId</code>)	구현완료 — zod 검증 + 소유권 체크(AC-006) 후 <code>prisma.creatorProfile.update . @MX:ANCHOR</code> 로 인가 불변식 명시

Auth (인증)

Method	Endpoint	기능	인증·역할	Request	Response	주요 오류	구현상태
GET	<code>/api/auth/[...nextauth]</code>	Auth.js v5 인증 핸들러(세션/CSRF/provider 콜백 등 catch-all)	공개 (NextAuth 내부 처리)	NextAuth 규약(엔드포인트별 상이)	NextAuth 표준 응답(세션 JSON·리다이렉트 등)	NextAuth 내부 처리(엔드포인트별 상이)	구현완료 — <code>@/auth</code> 의 <code>handlers</code> 위임. NextAuth({PrismaAdapter, JWT 세션, Credentials+Google(env 조건부)})
POST	<code>/api/auth/[...nextauth]</code>	Auth.js v5 인증 핸들러(로그인/로그아웃/콜백 등 catch-all)	공개 (Credentials authorize: 이메일·비밀번호 검증 후 발급)	NextAuth 규약 (Credentials: email/password)	NextAuth 표준 응답(세션 발급·리다이렉트 등)	NextAuth 내부 처리(자격 불일치 시 인증 실패)	구현완료 — <code>handlers.POST</code> 위임. <code>credentialsProvider.authorize</code> 에서 <code>prisma.user</code> 조회 + <code>verifyPassword</code> , OAuth 전용(passwordHash 없음) 계정은 비밀번호 로그인 차단

참고(전수 검증 메모)

- 세 도메인의 실제 `route.ts` 는 총 4개 파일·6개 HTTP 메서드 export가 전부다(빠진 라우트/메서드 없음). `studio` · `auth` 디렉터리에는 중첩 하위 라우트가 존재하지 않는다.
- `auth/[...nextauth]` 는 GET·POST 두 메서드만 export(`export const { GET, POST } = handlers`). PATCH/PUT/DELETE는 존재하지 않는다.
- 알림 라우트의 `request` 인자는 모두 미사용(`_request`)이라 본문 스키마가 없다.